CASHKARO / BANKKARO

EduLoans: explainable matching and document operations

A documented rule model, verification gates and two working builds for a complex education-loan workflow.

Role: PRD, rule-model and workflow author; AI-assisted prototyping | Period: March 2025–Present

Reported outcomes and scope

62 rules · 8 lenders · Documented policy model

23 states · Documented origination lifecycle

9h → 4h · Reported analyst effort per file

An operating result from career records, separate from proof that every scaffold feature was live.

The product problem

Different lenders require different eligibility checks and documents. Regional marksheets, scanned tax returns and joint statements also create uncertainty that a pure OCR flow cannot safely resolve. A confident interface can mislead users if the policy underneath it is incomplete.

My role

I authored the PRD and lender-rule model, chose the initial matching approach and used parallel builds to make the policy and experience reviewable. The portfolio describes a policy/chat prototype alongside a React, TypeScript and Supabase application scaffold.

What I built

The first version uses deterministic, per-lender eligibility gates and scoring. The model covers 62 rules across eight lender configurations, with scores clamped between 0 and 100 and assigned to five status tiers.

The 23-state workflow requires KAM verification before showing BRE results. Document requirements depend on both lender and applicant type. Classification, validation and verification include a human-review path for uncertain files.

The build deliberately separates policy correctness from application scaffolding. Its documented backlog includes missing lender thresholds, rules-of-record reconciliation and further checklist coverage; these are release work rather than completed outcomes.

• Student, distribution partner and KAM journeys share a common product shell.

• Premium-university exceptions, loan amount and income gates remain lender-specific.

• The document checklist follows the same policy source as the matching rules.

• AI tools are selected for scaffolding, reasoning, review and integration rather than treated as interchangeable.

A policy decision that could be inspected

I chose deterministic matching for the first release and deferred ML scoring until outcome data existed. I modelled lenders separately: eligibility gates first, then scoring, a 0–100 score boundary and five result tiers. The policy review included premium-university income exceptions, course factors and urgency adjustments. The same inputs and rule version were intended to produce the same answer.

Missing policy was treated as release work. I identified 13 threshold gaps across five lenders and required relationship-manager confirmation with a policy screenshot. The backlog made that a P0 restriction for affected lenders. I also recorded rule-consistency options across the master and two builds: a deploy-time sync with a checksum, or a rules-administration workflow. These were documented controls and launch conditions.

Designing for students, partners and reviewers

I used one conversational shell to explore three jobs: student capture, partner submission and bulk handling, and KAM queue review. Role-specific conversations and action sheets made those jobs concrete. A separate KAM dashboard later entered the roadmap as the review workload became clearer.

Document requirements came from the lender-by-applicant combination. Classification and extraction could assist a case, while uncertain regional marksheets, tax scans and joint statements had a KAM review path. Verification was a prerequisite for showing a lender match, protecting expectations before the result. The roadmap also included 32 unsecured checklist combinations and operational case views. Emergent, Lovable, Claude, Gemini and Antigravity were used for distinct prototype, scaffold, policy-review and repository tasks.

What changed

Two working repositories made the policy, prototype and application structure available for review. The broader origination programme reports processing effort falling from nine to four hours per education-loan file, with approximately □1,562 in labour value saved per file. Those are workflow-level results rather than separate performance tests of the rules engine, OCR or every prototype screen.

The lesson from this work

The design makes uncertainty visible at the point where it affects a decision. Deterministic matching, verification gates and human fallback are documented product choices; production readiness still depends on confirmed lender policies and consistent rules across the builds.

Visual references and walkthroughs

EduLoans: system architecture and the deterministic decisioning core

EduLoans: system architecture and the deterministic decisioning core

EduLoans: per-lender business-rule scoring flow

EduLoans: per-lender business-rule scoring flow

EduLoans: canonical lead-workflow state machine

EduLoans: canonical lead-workflow state machine

EduLoans: classify, validate and verify documents with a KAM-review fallback

EduLoans: classify, validate and verify documents with a KAM-review fallback

Documented workflow: Applicant intake → KAM verification → Lender gates and scoring → Explainable matching tiers → Document checks and human fallback

An authored explanation of the workflow described in this case study. This is a diagram, not a captured product screen.

View the online case study for the full image gallery and playable videos.

Evidence and attribution

The technical portfolio distinguishes working builds from open release blockers.

The repositories support inspection of the implementation and product decisions.

The 23 states are workflow states. Missing thresholds, checklist expansion, rule sync and separate KAM dashboard are backlog. Processing effort and labour-value figures belong to the broader origination workflow; production completeness and model quality are not independently demonstrated.